

Day 48



Arrow Functions Revisited:

Arrow functions are a shorter and cleaner way to write functions in JavaScript. They are written using the `=>` (arrow) syntax.

They are mostly used for:

- short functions
- callbacks
- array methods
- improving readability

Example: both `add` and `add2` are same.

```
1  const add = (a, b) => {  
2    |   return a + b;  
3  };  
4  |  
5  const add2 = (a,b) => a+b;
```

Example: arrow function with argument

```
8  const sayHello = name => `Hello, ${name}!`;  
9  
10 console.log(sayHello('Alice'));
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS E:\JavaScript\Day41-50\Day48> node main.js  
● Hello, Alice!
```

Example: passing two arguments to the arrow function

```
12  const sayHello = (name, greeting) => `${greeting}, ${name}!`;
13
14  console.log(sayHello('Alice', 'Welcome'));
15
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS E:\JavaScript\Day41-50\Day48> node main.js
● Welcome, Alice!
```

Example: no parameters

```
16  const greet = () => "Hello!";
17  console.log(greet());
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS E:\JavaScript\Day41-50\Day48> node main.js
● Hello!
```

Example: one arguments

```
18
19  const square = x => x * x;
20  console.log(square(5));
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS E:\JavaScript\Day41-50\Day48> node main.js
● 25
```

Example: arrow function don't have their own 'this', They use the **this** of the outer scope (lexical this). If we used a normal function inside setTimeout, it would NOT print the name correctly. Arrow functions fix that.

```
22  const person = {
23    name: "Aditya",
24    showName: function() {
25      setTimeout(() => {
26        console.log(this.name);
27      }, 1000);
28    }
29  };
30
31  person.showName(); // Aditya
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS E:\JavaScript\Day41-50\Day48> node main.js
Aditya

--The End--